```
    public List<ToDoListModel> Items { get; set; }        ❺
}
```

❶ The ToDoService is provided in the model constructor using DI.

❷ The OnGet handler takes a parameter, category.

❸ The handler calls out to the ToDoService to retrieve data and sets the Items property.

❹ Returns a PageResult indicating the Razor view should be rendered

❺ The Razor View can access the Items property when it's rendered.

This example is still relatively simple, but it demonstrates a variety of features compared with the basic example from listing 13.4:

- The page handler, `OnGet`, accepts a method parameter, `category`. This parameter is automatically populated using values from the incoming request via model binding, similar to the way binding works with minimal APIs. I discuss Razor Pages model binding in detail in chapter 16.
- The handler doesn't interact with the database directly. Instead, it uses the `category` value provided to interact with the `ToDoService`, which is injected as a constructor argument using DI.
- The handler returns `Page()` at the end of the method to indicate that the associated Razor view should be rendered. The `return` statement is optional in this case; by convention, if the page handler is a `void` method, the Razor view will still be rendered, behaving as though you called `return Page()` at the end of the method.
- The Razor View has access to the `CategoryModel` instance, so it can access the `Items` property that's set by the handler. It uses these items to build the HTML that is ultimately sent to the user.